

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**  
**КЕМЕРОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ**  
**Институт цифры**

**ОТЧЕТ**

**О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №5**

по дисциплине «Имитационное моделирование»

студента 4 курса, группы ФИТ-231

**Бабенко Данил Викторович**

направление подготовки 02.03.02 Фундаментальная информатика и  
информационные технологии

Руководитель:  
к.т.н., доцент кафедры прикладной  
математики  
Новосельцева М. А.  
Работа защищена  
«\_\_\_\_\_» \_\_\_\_\_ 20\_\_ г.

Кемерово 2026

## Задание 1

Смоделируйте время прибытия и количество прибывающих вагонов в течение  $T_n = 100$  часов. Сколько в среднем за сутки прибывало вагонов? Сравните с исходными данными. Сделайте графическую интерпретацию потока на временной оси.

Заданы параметры:

интенсивность потока поездов  $\lambda = 8/24$  поезда/час

среднее количество вагонов в поезде  $M = 10$

стандартное отклонение  $\sigma = 4$

время моделирования  $T = 100$  часов.

Интервал между поездами генерируется по экспоненциальному закону.  
Количество вагонов в поезде - по нормальному закону с округлением до целого

```

import math
import matplotlib.pyplot as plt
import numpy as np

lam = 8/24
Mx = 10
sigma = 4
T = 100
np.random.seed(37)
times = []
t = 0
current_time = 0

t_avg = Mx * lam * 24

while True:
    dt = -1 / lam * math.log(np.random.uniform(0, 1))
    t += dt
    if t > T:
        break
    times.append(t)

times = np.array(times)
n_trains = len(times)

wagons_raw = np.random.normal(Mx, sigma, n_trains)
wagons = np.round(wagons_raw).astype(int)
wagons = np.maximum(wagons, 0)
total_wagons = wagons.sum()
avg = total_wagons / T * 24

print(f"Промоделировано часов: {T}")
print(f"Количество поездов: {n_trains}")
print(f"Ожидалось поездов (теория): {lam * T:.2f}")
print(f"Вагонов: {total_wagons}")
print(f"Среднее число вагонов в поезде: {wagons.mean():.2f} (теория {Mx})")
print(f"Среднее число вагонов в сутки: {avg:.2f} (теория {lam * 24 * Mx:.2f})")

```

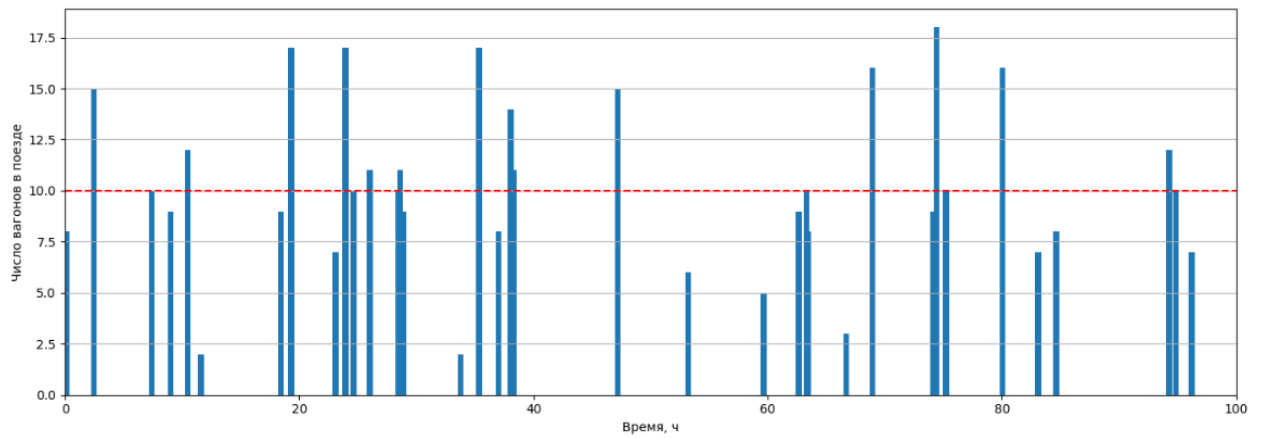
Вывод:

```

Промоделировано часов: 100
Количество поездов: 37
Ожидалось поездов (теория): 33.33
Вагонов: 378
Среднее число вагонов в поезде: 10.22 (теория 10)
Среднее число вагонов в сутки: 90.72 (теория 80.00)

```

Вывод:



## Задание 2:

Осуществите имитационное моделирование нестационарного неординарного потока запросов на сервер, задав интенсивность запросов в течение суток по часам.

Задана интенсивность запросов по часам суток (пики в 10-11 и 16-17, минимум ночью)

Время моделирования - 1 день

```

import numpy as np
import matplotlib.pyplot as plt
import math

np.random.seed(42)

lambda_hourly = np.array([1, 1, 1, 1, 2, 3, 5, 8, 12, 15, 18, 20, 20, 18, 17, 16, 15, 13, 10, 8, 6, 4, 3, 2], dtype=float)

T = 24
lam_max = lambda_hourly.max()
print(f"λ_max = {lam_max}")

def lam_of_t(t):
    h = int(t) % 24
    return lambda_hourly[h]

mean_dt = 1.0 / lam_max
t = 0.0
candidates = []
while True:
    dt = -mean_dt * math.log(np.random.uniform(0, 1))
    t += dt
    if t > T:
        break
    candidates.append(t)

accepted = []
for tc in candidates:
    p_accept = lam_of_t(tc) / lam_max
    if np.random.uniform(0, 1) < p_accept:
        accepted.append(tc)

accepted = np.array(accepted)
n_requests = len(accepted)
print(f"Число кандидатов: {len(candidates)}")
print(f"Принято запросов: {n_requests}")

Mx_w, sigma_w = 5, 2
units_raw = np.random.normal(Mx_w, sigma_w, n_requests)
units = np.maximum(np.round(units_raw).astype(int), 0)

total_units = units.sum()
print(f"Всего транзакций: {total_units}")

expected_requests = lambda_hourly.sum()
expected_units = expected_requests * Mx_w
print(f"Ожидалось запросов (теория): {expected_requests:.1f}")
print(f"Ожидалось транзакций (теория): {expected_units:.1f}")
print(f"Отношение имитация/теория: {n_requests / expected_requests:.3f}")

```

Вывод:

```

λ_max = 20.0
Число кандидатов: 460
Принято запросов: 230
Всего транзакций: 1135
Ожидалось запросов (теория): 219.0
Ожидалось транзакций (теория): 1095.0
Отношение имитация/теория: 1.050

```

